
MP9: Tessellation Shaders and Height Maps

Due Time: Refer to the course web-site

Objectives

Verify that we understand and can work with:

- Unity specific skills
 - There should be on new skills involved
- Shader and relevant general concepts:
 - Work with the tessellation and geometry shaders in the GPU pipeline
 - Implement geometry height changes as results of a cubic function
 - Implement geometry height changes as results of displacements from a texture map

Background

A cubic Bezier curve is with four control points, P_0, P_1, P_2 , and P_3 , where given $0 \leq t \leq 1$,

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

When the control points are position in 3D space, the resulting $B(t)$ is a well-defined cubic curve that intersects the end points, P_0 when $t = 0$, P_3 when $t = 1$, and generally does not intersect with P_1 and P_2 . Let's define $B_x(t)$ as the cubic Bezier curve where all control points are constrained to $z = 0$ with user manipulatable x, y positions. An intuitive way to interpret $B_x(t)$ is that it is a cubic curve in the X/Y space where the y -value varies as a cubic function of x .

Now, given a flat plane on the $y = 0$ plane with vertex position, $(v_x, 0, v_z)$, if we know the x -dimension bound, (min_x, max_x) , then, we can define t_x to be:

$$t_x = \frac{v_x - min_x}{max_x - min_x}$$

In this case, it is always true that, $0 \leq t_x \leq 1$, and thus, assigning the (x, y) values from $B_x(t_x)$, (a_x, a_y) , to the vertex positions **without altering** v_z , resulting in: (a_x, a_y, v_z) , would define a cubic plane varying in y along the x -axis with no changes in y along the z -direction.

Correspondingly, we can define $B_z(t_z)$ to be the curve with control points that are constrained to $x = 0$, t_z to be,

$$t_z = \frac{v_z - min_z}{max_z - min_z}$$

and thus assigning the (y, z) values from $B_z(t_z)$, (b_y, b_z) , to the vertex positions **without altering** v_x , resulting in: (v_x, b_y, b_z) , would define a cubic plane varying in y along the z -axis with no changes in y along the x -direction.

Lastly, if user can manipulate a variable, λ , that ranges between 0 and 1, then, given a vertex position, $(v_x, 0, v_z)$, we can linearly blend the two cubic function results by assigning:

- X coordinate to be: $\lambda a_x + (1 - \lambda) v_x$
- Y coordinate to be: $\lambda a_y + (1 - \lambda) b_y$
- Z coordinate to be: $(1 - \lambda) b_z + \lambda v_z$

In this way, when $\lambda = 1$, the results are governed by $B_x(t_x)$ and $B_z(t_z)$ when $\lambda = 0$.

Approach

You will support tessellation based on the discussed Bezier functions along X/Z axis. Additionally, you will support displacement based on texture map on the blended Bezier plane. Here are the details.

Bezier Plane

Please refer to the [MP9 Sample project](#). The Ref_Tess game object is the 10x10 Unity plane with two copies of CV_Profile.cs scripts and a single TessControl.cs script. Upon execution, the two CV_Profile will create a red and blue series of connected spheres. The red-series of spheres are the control points for $B_x(t_x)$, and the blue-series are for $B_z(t_z)$. Please refer to the CV_Profile.cs script, the positions of these control points are sent to shader. Note, shader variables are not defined.

Your job is to implement the discussed 2D Bezier blending. Please do use the provided Fragment Shader so that I can examine the proper components of your blended results. Here are additional controls you must support:

- Compute Bezier in X/Z: enable/disable the computation in X/Z directions
- XZ Blend: this is the λ blending factor discussed.
- Normal approximation distance: this is a *percentage* of the entire curve length, this is the delta-offset you will use to compute height difference for approximating new normals.

Displacement based on texture map

Additionally, you will support displacement of the blended Bezier plane based on a given texture map (`_MainTex`). You must ensure proper displacements along the new normals based on the gray color value of corresponding texels in the texture map. You must support the following controls:

- Compute displacement map: on/off switch to enable/disable the displacement computation
- Height Map Strength: a simple multiplier to scale the gray color value
- Height Map Level: which level of the mipmap to sample to obtain the height displacement
- Normal Approximation Map Level: when approximating new normal, which mipmap level to read from to compute the height differences in u and v directions.

Tessellation control

Lastly, you must support two tessellation modes, screen space and object space. In screen space tessellation, you should allow the user control the approximated number of pixels each triangle edge covers. In object space, you should allow the user control the approximated number of triangles edges to be used to represent the entire curve from $t=0$, to $t=1$. Controls you must support include:

- Screen/Object space tessellation enable
- Edge length in pixels when Screen Space is enabled
- Edges per Curve Length: number of edges should be used in representing the entire curve

Fragment shader

Please do not change the Fragment Shader provided, in particular, you must allow user the choice of displaying:

- Shaded/Unshaded texture
- UV
- Normal
- Light source direction
- NdotL

Hints

- My apologies, but, some of the variables in the shaders are defined, while others are not. Additionally, the variables are defined in separate files. I understand this can make things confusing. I am sorry,
- When working with debugging, make sure to articulate what you should observe and then display the results in Fragment Shader and verify.
- Bezier
 - Begin by implementing X (or Z) only, verify output is correct, including the normal before working on the second axis.

- When approximating the normal resulting from X/Z Bezier blending, make sure you remember to blend the y-changes along X and Z directions with λ and $1 - \lambda$. You can verify the normals by looking at NdotL in fragment shader and varying λ to observe smooth illumination changes.
 - Make sure to test your implementation by switching on/off X/Z computations.
- Texture map displacement:
 - Initially, disable the Bezier computations (that's why the controls are there!), verify all is working before switching on one of the Bezier axis to verify.
 - Make sure you understand normal computation based on tangent and binormal!
- Make sure your Fragment Shader is able display normal related information properly. Normal related credits will only be given when I can visual examine normal and NdotL results.

Credit Distribution	Credits
1 Unity Scene Setting: 5	
One plane with proper shader	5
MainTex is used for both color and displacement	
2 Displacement from Bezier blending: 35	
Control points control shape in X or Z	12
Blending support for X/Z	10
Proper normal approximation	8
Separate axis on/off control	5
3 Displacement from texture map: 30	
Basic displacement along new normal	6
Normal results are correct	10
Height strength	5
Height map level	6
Normal map approx level	3
4 Tessellation: 15	
Screen space on/off	2
Object space on/off	2
Screen space edge length	5
Object space edge length	6
5 Fragment Shader Displays	
Supports all listed	10
Show Normal and NdotL: 10	
6 Submission	
Proper folders/unity version	5
Total	100

This assignment counts 10% towards your final grade for this class.